Evolving Pedagogy: Applying Darwin-Gödel-Machine Principles to Self-Improving Intelligent Textbooks

A Framework for Empirically-Driven MicroSim Evolution
Using Learning Analytics

Dan McCreary
https://www.linkedin.com/in/danmccreary/

Working Paper — Draft v1.1
June 14, 2026

Abstract

Modern intelligent textbooks combine structured concept dependency graphs, learning-objective hierarchies, and interactive simulations ("MicroSims") to deliver adaptive instruction at scale. Yet the pedagogical artifacts within these textbooks remain static: once an author hand-crafts an explanation or simulation, it rarely changes in response to evidence about whether students actually learn from it. This paper proposes a framework for *self-improving intelligent textbooks* that adapts the core principles of the Darwin-Gödel-Machine (DGM)—empirical validation in place of formal proof, and open-ended archival evolution in place of single-solution optimization—to the domain of educational content. We map the three DGM mechanisms (self-modification, empirical fitness evaluation, and open-ended exploration) onto a concrete textbook architecture built on MkDocs, p5.js MicroSims, and a learning analytics pipeline using the Experience API (xAPI) [1] and a Learning Record Store (LRS). We situate the proposal within the rapidly advancing landscape of self-improving AI systems including the Darwin-Gödel-Machine [2], AlphaEvolve [3], and Self-Adapting Language Models (SEAL) [4]. Finally, we propose a detailed JSON schema standard for storing MicroSim performance data that bridges anonymous web analytics and identity-bearing xAPI streams, enabling a closed-loop evolutionary process for pedagogical content.

Keywords: intelligent textbooks, self-improving AI, Darwin-Gödel-Machine, MicroSims, learning analytics, xAPI, open-ended evolution, educational technology

1 Introduction

1.1 The Static-Content Problem

Educational content has historically been authored once and revised infrequently. A textbook chapter, a worked example, or an interactive diagram is designed using the author's best judgment about what will help students learn, and then deployed largely unchanged for years. This stands in stark contrast to how learning actually works: the effectiveness of any given explanation varies enormously across learners, contexts, and prior knowledge states.

The emergence of *intelligent textbooks*—interactive, AI-assisted learning materials that incorporate concept dependency graphs, learning-objective taxonomies, and embedded simulations—creates an opportunity to break this static mold. Because these textbooks are instrumented (they

can record how students interact with content) and modular (individual components can be regenerated independently), they are uniquely positioned to support *continuous, evidence-driven improvement* of their own pedagogical artifacts.

1.2 Learning from Self-Improving AI

A parallel revolution has been unfolding in artificial intelligence research. A class of systems has emerged that can improve their own performance through iterative self-modification, validated against empirical benchmarks rather than human design alone. The most relevant of these is the **Darwin-Gödel-Machine (DGM)**, introduced in 2025 by researchers at Sakana AI, the University of British Columbia, and the Vector Institute [2, 5].

The central insight of the DGM is a pragmatic relaxation. The original Gödel Machine, proposed theoretically by Jürgen Schmidhuber [6], required that any self-modification be *provably* beneficial before adoption—a requirement that is intractable for complex systems. The DGM instead requires only *empirical evidence* that a proposed modification improves performance on real benchmarks. This single shift transforms self-improvement from a theoretical impossibility into an engineering practice.

This paper argues that the same shift can be applied to educational content. We do not need to *prove* that a given MicroSim or explanation is pedagogically optimal. We need only gather empirical evidence—from learning analytics—that one variant outperforms another, and then let an archive of variants evolve over time. The remainder of this paper develops this analogy into a concrete, implementable framework.

1.3 Contributions

This paper makes the following contributions:

1. A conceptual mapping between DGM mechanisms and intelligent textbook architecture.
2. A survey of the current state of the art in self-improving AI systems relevant to education.
3. A worked architecture using MkDocs, p5.js MicroSims, and an xAPI/LRS analytics pipeline.
4. A proposed JSON schema standard for storing MicroSim performance data.
5. A discussion of safety constraints and human oversight requirements for evolving educational content.

2 Background

2.1 Intelligent Textbooks and MicroSims

An intelligent textbook differs from a traditional digital textbook in several structural ways. It is typically organized around a **concept dependency graph**—a directed acyclic graph in which nodes represent individual concepts and edges represent prerequisite relationships. Learning objectives are tagged against a taxonomy such as Bloom's Taxonomy [7], which orders cognitive demand from basic recall (*Remember*, *Understand*) through application and analysis (*Apply*, *Analyze*) to synthesis and judgment (*Evaluate*, *Create*).

The most distinctive feature is the **MicroSim**: a small, self-contained interactive simulation that lets a learner manipulate parameters and observe outcomes [8]. A MicroSim built with the

p5.js JavaScript library, for example, might let a student adjust the initial velocity of a bouncing ball and observe the resulting trajectory, building intuition for Newtonian mechanics. MicroSims are typically packaged as a directory containing a main HTML file, the simulation JavaScript, and supporting metadata, and are embedded into textbook pages via an inline frame.

A useful design principle for intelligent textbooks is the *five-level framework*, developed in detail by McCreary [9]. Table 1 presents the five levels, using the names established in that framework, together with two threshold columns that carry significant practical implications for institutional deployment.

Table 1: Five Levels of Intelligent Textbook Capability (names after [9])

Level	Name	Key Institutional Requirements	Student Data	LLM Required
1	Static	Fixed content only; no institutional systems required	No	No
2	Interactive	Static hosting; anonymous analytics optional	No	No
3	Adaptive	Student authentication; institutional data infrastructure; ongoing relationship management	Yes	No
4	Chatbot	Integration with institutional systems; LLM oversight capabilities; trust relationships	Yes	Yes
5	Autonomous	Deep institutional knowledge; comprehensive student support systems	Yes	Yes

Two threshold boundaries emerge from Table 1 with significant practical implications. The first is a **data-privacy boundary** between Levels 2 and 3: Levels 1 and 2 require no student-specific data and can be served to completely anonymous users, while the Adaptive level (Level 3) and above demand student authentication, individual learning histories, assessment records, and ongoing institutional relationship management. This boundary triggers obligations under data-protection regulations such as FERPA, COPPA, and GDPR, and it is the threshold below which institutions can deploy intelligent textbooks with minimal governance overhead.

The second is an **LLM-cost boundary** between Levels 3 and 4. Levels 1 through 3 can be implemented entirely with deterministic algorithms—graph traversal, rule-based sequencing, and spaced repetition—without any large language model inference. A well-constructed concept dependency graph, combined with per-learner mastery estimates, is sufficient to drive adaptive navigation without generative AI. The Chatbot level (Level 4) and the Autonomous level (Level 5), by contrast, require continuous LLM access—Level 4 for real-time conversational tutoring and Level 5 for autonomous content generation and self-modification. This dependency increases operating costs by one to two orders of magnitude relative to lower levels and introduces ongoing dependencies on third-party model providers.

This paper is fundamentally concerned with the transition from the Chatbot level (Level 4) to the Autonomous level (Level 5)—the point at which the textbook begins to improve its own content based on empirical evidence from learning analytics. Because Level 5 inherits the LLM dependency of Level 4 and adds the cost of continuous fitness evaluation and variant generation, the

evolutionary loop proposed here must account for inference cost as a component of overall system viability.

2.2 The Darwin-Gödel-Machine

The Darwin-Gödel-Machine is a self-improving system that iteratively modifies its own code while empirically validating each change against coding benchmarks [2]. Its design draws on two intellectual traditions. From the theoretical Gödel Machine it inherits the goal of self-referential improvement, but it replaces the requirement of formal proof with empirical benchmark validation. From Darwinian evolution and open-endedness research it inherits the practice of maintaining an *archive* of generated solutions rather than evolving a single line of descent.

The system operates in an iterative cycle. It begins with a base coding agent built on a frozen foundation model equipped with tool-use capabilities. In each generation, a parent agent is selected from the archive, the parent is tasked with modifying itself to produce a child variant, the child is evaluated on benchmarks, and successful children are added to the archive. The archive enables open-ended exploration: previously suboptimal designs can serve as stepping stones to later breakthroughs, and many distinct improvement paths can be explored in parallel.

Reported results are substantial. The DGM improved its performance on the SWE-bench coding benchmark from 20.0% to 50.0%, and on the Polyglot multi-language benchmark from 14.2% to 30.7%. Ablation studies demonstrated that both self-modification and open-ended exploration were essential; disabling either substantially degraded performance. The authors emphasize that all experiments were conducted with safety precautions including sandboxing and human oversight.

Three properties of the DGM are directly relevant to educational content:

1. **Empirical over formal validation.** We cannot prove a MicroSim is pedagogically optimal, but we can measure whether students learn better from one variant than another.

2. **Archival open-endedness.** Maintaining a population of content variants avoids premature convergence on a locally good but globally suboptimal design.

3. **Frozen foundation, evolving scaffold.** The DGM does not modify its underlying model; it modifies the code and tools around it. Analogously, an educational system need not modify the underlying language model—only the content artifacts and the prompts that generate them.

2.3 The Broader Landscape of Self-Improving Systems

The DGM is one of several recent systems exploring self-improvement, and understanding the landscape clarifies which mechanisms are most applicable to education.

AlphaEvolve [3] (Google DeepMind, May 2025) is an evolutionary coding agent that pairs large language models with evolutionary computation to discover and refine algorithms. It orchestrates an autonomous pipeline of LLMs that propose direct changes to code, with one or more automatic evaluators providing feedback that grounds the evolutionary process. AlphaEvolve requires an evaluation function with metrics to optimize and an initial algorithm to seed the search. Among its notable results, it discovered a procedure to multiply two 4×4 complex-valued matrices using 48 scalar multiplications, the first improvement over Strassen's 1969 algorithm in that setting, and it produced practical optimizations to data center scheduling and hardware accelerator circuit design. AlphaEvolve's reliance on an explicit, automatic evaluation function is the single most transferable idea for educational content: the evaluator *is* the fitness function, and designing it well is the central challenge.

SEAL (Self-Adapting Language Models) [4] (MIT, June 2025; expanded for NeurIPS 2025) takes a different approach, enabling a model to generate its own fine-tuning data and update directives. The model produces "self-edits"—natural-language descriptions of what should change—which are converted into fine-tuning examples and applied via supervised learning, with a reinforcement learning outer loop that uses downstream task performance as the reward signal. On a factual question-answering task, SEAL improved accuracy from 32.7% to 47.0% after two rounds of training, outperforming fine-tuning on raw passages or on synthetic data generated by a strong external model. SEAL's relevance to education is its focus on *knowledge incorporation*—the same problem a textbook faces when it must integrate new material—and its explicit acknowledgment that human supervision remains necessary to ensure improvements are beneficial and aligned.

Open-source successors have followed quickly. CodeEvolve [10] reproduces the high-level principles of AlphaEvolve in an open framework using an island-based genetic algorithm with a weighted LLM ensemble, reporting competitive performance at a fraction of the compute cost. The Gödel Agent [11] sketches a self-referential architecture in which an agent proposes and accepts modifications to itself based on environmental feedback.

Table 2 summarizes the landscape.

Table 2: Landscape of Self-Improving AI Systems and Applicability to Textbooks

System	Organization	What evolves	Validation signal	Textbook applicability
Gödel Machine	Schmidhuber (theory)	Own code	Formal proof	Low (intractable)
Darwin-Gödel-Machine	Sakana AI / UBC / Vector	Agent code + tools	Coding benchmarks	High (archival model)
AlphaEvolve	Google DeepMind	Algorithms / code	Automatic evaluators	High (fitness function)
SEAL	MIT	Model weights	Downstream task RL reward	Medium (knowledge incorporation)
CodeEvolve	Open source	Code	Benchmark evaluators	High (reproducible, low cost)

The consistent pattern across all successful systems is the combination of a generative engine (a foundation model proposing variants) with a grounding evaluator (an automatic measure of quality). For educational content, the generative engine already exists in the form of code-generation tools; the missing piece is a well-designed evaluator built from learning analytics. The rest of this paper focuses on building that evaluator.

3 A Framework for Self-Improving Intelligent Textbooks

3.1 The Core Analogy

The mapping between DGM components and intelligent textbook components is direct enough to be productive. Table 3 makes it explicit.

Table 3: DGM–Intelligent Textbook Component Mapping

DGM Concept	Intelligent Textbook Analog
Coding agent codebase	MicroSim source files and content-generation skills
Frozen foundation model	The code-generation language model (unchanged)
Benchmark (SWE-bench)	Student learning outcomes and engagement metrics
Archive of agents	Versioned library of MicroSim and explanation variants
Parent selection	Choosing a high-performing variant to mutate
Self-modification	Generating a content variant with a pedagogical hypothesis
Fitness evaluation	Aggregating analytics into a per-variant score
Open-ended exploration	Concept-graph traversal and Bloom's-level laddering

3.2 The Evolutionary Loop

The proposed system operates in a generational cycle analogous to the DGM's, but with pedagogical artifacts as the unit of evolution:

1. **Selection.** From the archive of content variants for a given concept node, select a parent. Selection favors variants with strong measured outcomes but reserves probability mass for under-explored variants, balancing exploitation against exploration just as the DGM balances high-performing parents against those with few children.

2. **Mutation.** A code-generation tool produces one or more child variants of the parent. Each mutation is driven by an explicit *pedagogical hypothesis*—for example, "adding a real-time graph of velocity over time will improve student ability to connect the simulation to the underlying equation," or "reducing the number of simultaneous parameters from four to two will reduce cognitive load for novices."

3. **Deployment.** Child variants are deployed to a subset of learners, either through randomized assignment (A/B testing) or through a multi-armed bandit allocation that shifts traffic toward better-performing variants over time.

4. **Evaluation.** Learning analytics are gathered for each variant and aggregated into a fitness score (Section 3.4).

5. **Archival.** Variants meeting a quality threshold are retained in the archive with their full performance record, becoming candidate parents for future generations. Crucially, even underperforming variants are retained as potential stepping stones, preserving the open-endedness that the DGM ablation studies showed to be essential.

3.3 The Concept Dependency Graph as Search Space

In the DGM, the search space is the space of all computable agent programs. In an intelligent textbook, the concept dependency graph provides a far more structured and tractable search space. Each node is an independent evolutionary population. The graph topology supplies three useful signals:

- **Prioritization.** Concepts where students show weak mastery, or whose downstream dependents show degraded performance, are prioritized for evolutionary effort.
- **Diagnosis.** When learners consistently fail at a particular prerequisite edge, the system can hypothesize that a *bridging concept* is missing and propose inserting a new node—a structural mutation of the graph itself, not merely of content within a node.
- **Credit assignment.** Because the graph encodes prerequisites, the system can partially attribute a student's failure on a downstream concept to weaknesses in upstream content, focusing improvement where it will have the greatest cascading benefit.

3.4 The Fitness Function: From Analytics to Selection Pressure

The single most important design decision, as the AlphaEvolve experience makes clear, is the evaluator. A naive fitness function—for example, raw quiz score—invites perverse outcomes such as variants that make quizzes easier rather than instruction better. A robust pedagogical fitness function should be multi-dimensional and aligned with genuine learning.

We propose a composite fitness function with the following components:

- **Learning gain.** The improvement in assessment performance from before to after engaging with the MicroSim, normalized by the maximum possible gain. This is the primary signal and should dominate.
- **Bloom's-level advancement.** Whether the student demonstrates mastery at higher cognitive levels (*Apply*, *Analyze*) rather than mere recall. A variant that moves students up the taxonomy is worth more than one that merely improves recall.
- **Efficiency.** The number of attempts and the time required to reach mastery. Faster mastery, all else equal, indicates clearer instruction.
- **Engagement quality.** Interaction depth with the MicroSim, distinguishing meaningful exploration from disengagement or random clicking.
- **Retention.** Performance on the concept in later sessions, capturing durable learning rather than transient performance.

These components are combined into a single scalar with weights that reflect institutional priorities, with learning gain and retention weighted most heavily to guard against optimizing for shallow proxies. The composite design directly echoes AlphaEvolve's requirement for an evaluation function with explicit metrics, adapted to the multi-objective reality of learning.

3.5 Two Deployment Phases

A practical deployment must account for the reality that most intelligent textbooks begin life on a static host such as GitHub Pages with anonymous users, and only later integrate identity-bearing learning analytics. The framework supports both phases.

Phase 1: Anonymous aggregate analytics. With static hosting and no learner identity, the system relies on aggregate event analytics (for example, a web analytics platform) and browser-local storage for within-session continuity. This yields aggregate per-variant fitness signals sufficient to drive selection at the population level, though it cannot reconstruct individual learning trajectories.

Phase 2: Identity-bearing xAPI and LRS. As the textbook integrates the Experience API and a Learning Record Store [1], each interaction is recorded as a structured statement carrying a learner identifier, enabling per-learner trajectory analysis, retention measurement across sessions, and far richer fitness signals.

The critical engineering recommendation is to **design the Phase 1 analytics schema to match the Phase 2 xAPI schema from the outset**, using identical field names and concept identifiers. This makes the eventual migration a matter of swapping the data sink rather than re-instrumenting content, and allows fitness queries to run against either backend with minimal change.

4 Worked Examples

4.1 Example MicroSim: A Bouncing Ball for Newtonian Mechanics

Consider a MicroSim that visualizes a bouncing ball under gravity, targeting the concept *Newton's Second Law* within a physics textbook. The base variant exposes a single control: initial drop height. Several evolutionary hypotheses can be generated and tested:

- **Variant A (graph overlay):** Add a real-time plot of velocity versus time alongside the animation. *Hypothesis: connecting the visual motion to its quantitative representation improves transfer to symbolic problem-solving.*
- **Variant B (parameter reduction):** For novice learners, hide all controls except one, reducing extraneous cognitive load. *Hypothesis: fewer simultaneous variables improves initial conceptual grasp.*
- **Variant C (prediction prompt):** Before running the animation, ask the learner to predict where the ball will be after one second. *Hypothesis: active prediction (a generative learning strategy) improves retention.*

Each variant is deployed, its composite fitness measured, and the winner promoted. Note that Variant C tests a pedagogical strategy (prediction-based active learning) that generalizes across many MicroSims; a win here might propagate as a mutation operator applied to other concepts, mirroring how the DGM discovered general-purpose improvements such as solution-ranking that transferred across tasks.

4.2 Example MicroSim: Compound Interest for Personal Finance

A compound interest visualizer in a personal finance textbook lets learners adjust principal, interest rate, and time horizon, and observe the resulting growth curve. This MicroSim illustrates an important archival principle: a variant that performs poorly for one audience may be the ideal

stepping stone for another. A visually dense variant showing multiple overlaid scenarios might overwhelm high-school learners but excel for adult learners with financial background. The archive retains both, and selection becomes audience-conditioned—a natural extension of the concept-graph search space into a learner-segment dimension.

4.3 Example Textbook Domains

The framework applies across the disciplinary breadth typical of an intelligent-textbook portfolio. MicroSim-rich quantitative domains such as physics and biology at the secondary and International Baccalaureate level are especially strong candidates, because the abundance of simulatable phenomena gives the evolutionary process a large space of meaningful mutations. Computing curricula such as algorithms and data structures are equally promising: an animated sorting or graph-traversal MicroSim has many tunable pedagogical dimensions (animation speed, step annotation, comparison highlighting) that map naturally onto evolutionary parameters. History and other narrative domains benefit less from simulation but can still evolve their explanatory text and concept-graph structure.

4.4 Skill-Level Self-Improvement

A subtle but powerful application targets not the MicroSims themselves but the *content-generation skills* that produce them. Following the DGM's self-referential principle—that improving the codebase also improves the ability to improve the codebase—the prompts and templates used to generate MicroSims can themselves be treated as an evolving population. Variant prompts are scored by the average fitness of the MicroSims they produce, and high-performing prompts are retained and mutated. Over time, the generator improves, a genuine second-order effect that compounds the gains from content-level evolution.

5 Proposed JSON Standard for MicroSim Performance Data

This section proposes a concrete JSON schema for storing MicroSim performance data. The design has three goals: to capture the fields necessary to compute the fitness function of Section 3.4; to align with the Experience API so that the same instrumentation works in both deployment phases; and to support the archival, variant-centric data model that the evolutionary loop requires.

5.1 Design Principles

The schema follows several principles drawn from the xAPI and cmi5 [12] specifications and from the requirements of the evolutionary loop:

- **Variant-centric.** Every record identifies the specific MicroSim variant, not merely the concept, so that fitness can be computed per variant. This is the field that makes evolution possible and is the most important addition over conventional analytics.
- **xAPI-aligned.** Field names and structure mirror the xAPI Actor–Verb–Object–Result pattern. The xAPI specification recommends using built-in properties (such as the Result score) before resorting to custom extensions, since custom extensions may be excluded from standard LRS reports; the schema follows this guidance.

- **Standard verbs.** The schema uses registered xAPI and cmi5 verbs such as `initialized`, `interacted`, `answered`, `completed`, `passed`, and `failed` rather than inventing new ones, preserving interoperability.
- **Privacy-conscious.** Learner identity is carried in an account-based identifier rather than personal information, and the schema supports a fully anonymous mode for Phase 1.

5.2 The MicroSim Event Record Schema

The following is the proposed per-event record. It can be emitted directly to a web analytics platform in Phase 1 (flattened) or wrapped in a full xAPI statement in Phase 2.

```
{
  "$schema": "https://intelligenttextbooks.org/schemas/microsim-event/v1.json",
  "event_id": "a7f3c918-b88b-4220-90a5-a4c32391a240",
  "schema_version": "1.0.0",
  "timestamp": "2026-06-14T15:42:08.000Z",

  "actor": {
    "mode": "identified",
    "account_id": "student-uuid-or-anonymous-session-hash",
    "segment": "ib-physics-sl"
  },

  "verb": {
    "id": "http://adlnet.gov/expapi/verbs/completed",
    "display": "completed"
  },

  "microsim": {
    "sim_id": "bouncing-ball",
    "variant_id": "bouncing-ball-v3-graph-overlay",
    "parent_variant_id": "bouncing-ball-v2",
    "generation": 3,
    "hypothesis": "Real-time velocity graph improves transfer to symbolic problems",
    "source_url": "https://example.github.io/sims/bouncing-ball-v3/main.html"
  },

  "concept": {
    "concept_id": "newtons-second-law",
    "graph_node": "phys.mechanics.dynamics.n2l",
    "bloom_level": 2,
    "bloom_label": "Apply"
  },

  "result": {
    "score": {
      "scaled": 0.75,
      "raw": 15,
      "min": 0,
      "max": 20
    },
    "success": true,
    "completion": true,
    "duration_iso8601": "PT2M22S",
    "duration_seconds": 142
  },
```

```
  "interaction": {
    "attempts": 2,
    "param_changes": 11,
    "controls_used": ["drop_height", "gravity"],
    "idle_seconds": 8,
    "engagement_depth": 0.82
  },

  "learning": {
    "pre_assessment_score": 0.40,
    "post_assessment_score": 0.75,
    "normalized_gain": 0.58,
    "prediction_made": true,
    "prediction_correct": false
  },

  "context": {
    "textbook_id": "ib-physics-sl",
    "chapter": "dynamics",
    "registration": "ec231277-b27b-4c15-8291-d29225b2b8f7",
    "platform": "mkdocs-material",
    "client": "web",
    "ab_test_group": "treatment-graph-overlay"
  }
}
```

5.3 Field Reference

The schema's blocks correspond to the components of the fitness function and to xAPI's statement structure:

- **`actor`** carries learner identity. The `mode` field switches between `anonymous` (Phase 1) and `identified` (Phase 2). The `segment` enables audience-conditioned evolution as described in Section **??**.
- **`verb`** uses a registered xAPI verb identifier and display string, ensuring LRS interoperability.
- **`microsim`** is the variant-centric core. The `variant_id`, `parent_variant_id`, and `generation` fields encode the evolutionary lineage directly, allowing the archive to be reconstructed as a tree. The `hypothesis` field documents *why* the variant was created, paralleling the DGM's practice of recording what has been tried and why.
- **`concept`** ties the event to the dependency graph and Bloom's level, supplying the structured search-space coordinates.
- **`result`** uses xAPI's standard `score` object (scaled, raw, min, max), `success`, `completion`, and ISO 8601 duration, in keeping with the recommendation to prefer built-in properties.
- **`interaction`** captures engagement-quality signals that distinguish meaningful exploration from disengagement.
- **`learning`** carries the pre/post assessment and normalized-gain fields that drive the primary fitness component.
- **`context`** supplies textbook, chapter, A/B group, and the cmi5-style `registration` identifier that groups statements within a learning session.

5.4 The Aggregated Variant Fitness Record

Individual events are periodically aggregated into a per-variant fitness record. This is the structure the selection step reads from, and it serves as the sidecar metadata stored alongside each MicroSim variant in the archive.

```
{
  "$schema": "https://intelligenttextbooks.org/schemas/variant-fitness/v1.json",
  "variant_id": "bouncing-ball-v3-graph-overlay",
  "concept_id": "newtons-second-law",
  "generation": 3,
  "parent_variant_id": "bouncing-ball-v2",
  "hypothesis": "Real-time velocity graph improves transfer to symbolic problems",
  "window": {
    "start": "2026-05-01T00:00:00Z",
    "end": "2026-06-01T00:00:00Z"
  },
  "sample": {
    "n_learners": 412,
    "n_events": 1187,
    "segments": {
      "ib-physics-sl": 318,
      "us-high-school": 94
    }
  },
  "metrics": {
    "avg_normalized_gain": 0.58,
    "avg_post_score": 0.75,
    "bloom_advancement_rate": 0.41,
    "avg_attempts": 2.1,
    "avg_time_seconds": 138,
    "avg_engagement_depth": 0.79,
    "retention_score": 0.68,
    "completion_rate": 0.91
  },
  "fitness": {
    "composite_score": 0.712,
    "weights": {
      "normalized_gain": 0.35,
      "bloom_advancement": 0.20,
      "efficiency": 0.10,
      "engagement_quality": 0.10,
      "retention": 0.25
    },
    "confidence_interval_95": [0.681, 0.743],
    "rank_within_concept": 1
  },
  "status": "active",
  "human_review": {
    "approved": true,
    "reviewer": "content-team",
    "reviewed_at": "2026-05-02T10:15:00Z"
  }
}
```

The `fitness` block records both the composite score and the weights used to compute it, so that scoring remains auditable and reproducible as priorities change. The `confidence_interval_95` field is essential: selection should account for statistical uncertainty, preferring variants whose

advantage is well-established over those with a high but noisy point estimate. The `human_review` block enforces the oversight requirement discussed in Section 6; a variant cannot reach `active` status without review.

5.5 The Full xAPI Statement Form (Phase 2)

For Phase 2, the event record maps onto a conformant xAPI statement. The MicroSim-specific fields move into result and context extensions, with custom extension identifiers under an organizational namespace, while standard fields use built-in properties [13]:

```
{
  "id": "a7f3c918-b88b-4220-90a5-a4c32391a240",
  "actor": {
    "objectType": "Agent",
    "account": {
      "homePage": "https://example.github.io",
      "name": "student-uuid"
    }
  },
  "verb": {
    "id": "http://adlnet.gov/expapi/verbs/completed",
    "display": { "en-US": "completed" }
  },
  "object": {
    "id": "https://example.github.io/sims/bouncing-ball-v3",
    "objectType": "Activity",
    "definition": {
      "name": { "en-US": "Bouncing Ball MicroSim v3 (graph overlay)" },
      "type": "http://adlnet.gov/expapi/activities/simulation",
      "extensions": {
        "https://intelligenttextbooks.org/xapi/ext/variant-id":
            "bouncing-ball-v3-graph-overlay",
        "https://intelligenttextbooks.org/xapi/ext/concept-id":
            "newtons-second-law",
        "https://intelligenttextbooks.org/xapi/ext/bloom-level": 2
      }
    }
  },
  "result": {
    "score": { "scaled": 0.75, "raw": 15, "min": 0, "max": 20 },
    "success": true,
    "completion": true,
    "duration": "PT2M22S",
    "extensions": {
      "https://intelligenttextbooks.org/xapi/ext/normalized-gain": 0.58,
      "https://intelligenttextbooks.org/xapi/ext/attempts": 2,
      "https://intelligenttextbooks.org/xapi/ext/engagement-depth": 0.82
    }
  },
  "context": {
    "registration": "ec231277-b27b-4c15-8291-d29225b2b8f7",
    "contextActivities": {
      "category": [
        { "id": "https://w3id.org/xapi/cmi5/context/categories/moveon" }
      ],
      "parent": [
        { "id": "https://example.github.io/textbooks/ib-physics-sl/dynamics" }
      ]
```

```
    }
  },
  "timestamp": "2026-06-14T15:42:08.000Z"
}
```

This statement is conformant with the xAPI data model and follows cmi5 conventions for category context activities and the registration identifier, so it can be stored in any standards-compliant Learning Record Store and queried by the fitness-aggregation pipeline.

6 Safety, Oversight, and Ethical Considerations

The DGM authors are explicit that their experiments relied on sandboxing and human oversight, and they note that the broader safety challenges of recursively self-improving systems remain open [2]. SEAL's authors similarly emphasize that human supervision is required to ensure improvements remain beneficial and aligned [4]. Evolving educational content raises its own distinct concerns that demand comparable care.

Goodhart's Law and proxy gaming. Any fitness function is a proxy for genuine learning, and optimizing a proxy can degrade the true objective. A variant that maximizes quiz scores by narrowing instruction to test items, or that maximizes engagement through manipulative design, would score well while harming learning. The multi-dimensional fitness function of Section 3.4, with heavy weight on retention and normalized gain rather than raw scores, is a partial defense, but human review of *why* a variant performs well remains essential.

Human-in-the-loop gating. No generated variant should reach learners without review. The `human_review` block in the fitness schema enforces this structurally: a variant cannot become `active` without an approving reviewer. This mirrors the DGM's fixed, human-controlled parent-selection and archiving process, which the authors describe as an implicit safety constraint.

Equity across learner segments. Because the framework supports audience-conditioned evolution, it risks optimizing well for majority segments while neglecting smaller ones. Fitness aggregation should monitor per-segment performance and flag variants that improve aggregate scores while harming any segment, particularly underserved populations such as rural or under-resourced learners.

Privacy. The schema deliberately uses account-based identifiers rather than personal information and supports a fully anonymous mode. Learner data feeding the evolutionary loop should be governed by clear consent and data-minimization practices, and identifiers should never be embedded in URLs or query strings.

Sandboxed deployment. Generated MicroSim code should be validated in an isolated preview environment before any public deployment, a practice that aligns naturally with a preview-branch workflow on a static host.

7 Discussion and Future Work

The framework proposed here is, at present, a design rather than a validated system, and several questions remain open. The most pressing is empirical: does evolutionary refinement of MicroSims actually produce measurable learning gains over hand-authored content, and at what cost? The DGM and AlphaEvolve results in their respective domains are encouraging, but education is a noisier domain with smaller effect sizes and far greater measurement difficulty, so direct transfer of their success rates should not be assumed.

Several avenues merit investigation. Transferable mutation operators—pedagogical strategies such as prediction prompts that win in one concept and propagate to others—could dramatically accelerate the search, echoing the general-purpose improvements the DGM discovered. The graph-structural mutations of Section 4.3, in which the system proposes new bridging concepts, represent a more ambitious form of self-improvement that modifies the search space itself. And the skill-level self-improvement of Section 4.4 raises the possibility of compounding second-order gains, though it also compounds the safety concerns and would require correspondingly stronger oversight.

The convergence of three trends makes this an opportune moment for such work: foundation models capable of generating high-quality educational content [14], mature learning-analytics standards in xAPI and cmi5 that provide the empirical grounding, and a body of demonstrated self-improvement systems that provide a proven architectural template. The central thesis of this paper is that intelligent textbooks possess all the structural ingredients—modular content, instrumented delivery, and a graph-organized search space—to become, in the DGM's phrase, capable of gathering their own stepping stones along a path that unfolds toward continuously improving instruction.

8 Conclusion

We have argued that the principles underlying the Darwin-Gödel-Machine—empirical validation in place of formal proof, and open-ended archival evolution in place of single-solution optimization—can be productively transferred from the domain of self-improving code agents to the domain of intelligent textbooks. We mapped the DGM's mechanisms onto a concrete architecture, situated the proposal within the current landscape of self-improving systems including AlphaEvolve and SEAL, illustrated the approach with worked MicroSim examples, and proposed a detailed, xAPI-aligned JSON standard for storing the MicroSim performance data that makes the evolutionary loop possible.

The framework is designed to operate across two deployment phases, from anonymous aggregate analytics on static hosting to identity-bearing xAPI streams in a Learning Record Store, and it embeds human oversight as a structural requirement rather than an afterthought. Whether evolved pedagogy can match or exceed hand-authored content is ultimately an empirical question, and the schema and architecture proposed here are intended to make that question answerable.

This is a working draft intended to stimulate discussion and guide implementation. All quantitative results attributed to external systems (DGM, AlphaEvolve, SEAL) are drawn from their respective published sources; the intelligent-textbook framework and JSON schema proposed herein are design proposals that have not yet been empirically validated.

References

[1] ADL Initiative, "Experience API (xAPI) specification." GitHub, `https://github.com/adlnet/xAPI-Spec`, 2013.

[2] J. Zhang, S. Hu, C. Lu, R. Lange, *et al.*, "Darwin gödel machine: Open-ended evolution of self-improving agents." arXiv:2505.22954, 2025.

[3] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery." arXiv:2506.13131, 2025.

[4] A. Zweiger, J. Pari, *et al.*, "Self-adapting language models (SEAL)." arXiv:2506.10943, NeurIPS 2025, 2025.

[5] Sakana AI, "The darwin gödel machine: AI that improves itself by rewriting its own code." Sakana AI Blog, `https://sakana.ai/dgm/`, 2025.

[6] J. Schmidhuber, "Gödel machines: Fully self-referential optimal universal self-improvers," in *Artificial General Intelligence*, Springer, 2007.

[7] B. S. Bloom *et al.*, "Taxonomy of educational objectives: The classification of educational goals," *Handbook I: Cognitive Domain*, 1956.

[8] V. Lockhart, D. McCreary, and T. A. Peterson, "MicroSims: A framework for AI-generated, scalable educational simulations with universal embedding and adaptive learning support," *arXiv preprint arXiv:2511.19864*, 2025.

[9] D. McCreary, "A five-level classification framework for intelligent textbooks: Lessons from autonomous vehicle standards." EdArXiv Preprint, `https://osf.io/preprints/edarxiv/sh2yu_v1`, December 2025. Version 0.04.

[10] H. S. Assumpção *et al.*, "CodeEvolve: An open source evolutionary coding agent for algorithmic discovery and optimization." arXiv:2510.14150, 2025.

[11] Yin *et al.*, "Gödel agent: A self-referential agent framework." Preprint, 2024.

[12] AICC and ADL, "cmi5 specification." GitHub, `https://github.com/AICC/CMI-5_Spec_Current`, 2015.

[13] Rustici Software, "cmi5 technical overview and xAPI statements 101." `https://xapi.com/cmi5/technical/` and `https://xapi.com/statements-101/`, 2024.

[14] D. McCreary, "Claude skills for intelligent textbooks." `https://dmccreary.github.io/claude-skills/`, 2025.